Last updated on ***Apr 15, 2026***

Extend Genie Code with agent skills

Genie Code comes with built-in skills that are pre-configured for common Databricks workflows, such as writing code in Databricks notebooks, exploring data in Unity Catalog, building dashboards, creating pipelines, and working with MLflow. You can also create your own skills to extend Genie Code in Agent mode with specialized capabilities for your domain-specific tasks. This page explains how to create and optimize skills.

What are skills?

Create skills to extend Genie Code with specialized capabilities. Skills follow the open standard of Agent Skills. Skills package domain-specific knowledge and workflows that Genie Code can load when relevant to perform specific tasks. Skills can include guidance, best practices, reusable code, and executable scripts.

Skills should be tailored for domain-specific tasks. With skills, you can provide greater context (such as scripts, examples, and other resources) for a task than you can with instructions. Unlike custom instructions, which are applied globally, skills are loaded automatically and only in the relevant context. In Agent mode, Genie Code automatically loads skills when relevant, based on your request and the skill's description. You can also manually invoke skills by `@` mentioning them. This maintains an efficient context window and reduces the need to provide the same context across multiple chats.

There are two types of skills:

- **Workspace skills**: Available to everyone in the workspace. Workspace admins can create workspace skills and grant others access to the skills folder to add more. Use workspace skills for workflows that are broadly useful across your team, such as machine learning workflows or domain-specific processes.
- **User skills**: Available only to you. Use user skills for personal workflows that are not relevant to other workspace members.

> **NOTE**
>
> Skills are only supported in Genie Code Agent mode.

Create a skill

Skills live in a `.assistant/skills/` directory. Each skill must have its own folder and a `SKILL.md` file within that folder. The location depends on the skill type:

- **Workspace skills**: `Workspace/.assistant/skills/`
- **User skills**: `/Users/{username}/.assistant/skills/`

To create a new skill:

1. Create a new skills folder at the appropriate path for your skill type.

   After creation, you can quickly access your skills folder in Genie Code panel. Click **Settings**, then click **Open skills folder**.

2. Create a dedicated folder for your skill inside the skills folder. Each skill must have its own folder. For example:

```
Workspace/.assistant/skills/
└── ml-workflows/
    └── SKILL.md

/Users/{username}/.assistant/skills/
└── personal-workflows/
    └── SKILL.md
```

3. Inside your skill folder, create a `SKILL.md` file. This file is required and defines the skill. Skills follow the specifications of Agent Skills.

4. Add the required frontmatter for your skill:

```
---
name: skill-name
description: A description of what this skill does and when to use it.
---
```

5. Add the skill instructions in Markdown format after the frontmatter. It's recommended to include the following sections:
   - Step-by-step instructions: Clear procedural guidance
   - Examples: Sample inputs and expected outputs
   - Edge cases: Common variations and exceptions
6. (Optional) For more complex skills, you can provide and reference additional resources:
   - Scripts containing executable code that the agent can run.
   - Files containing additional documentation to reference, such as best practices and templates.

   When referencing other files, use relative paths from the root skill.

   For example, a workspace machine learning workflow skill and a personal workflow skill might have the following structure:

```
Workspace/.assistant/skills/
├── ml-workflows/
   ├── SKILL.md                # Workflow overview and best practices
   ├── training-patterns.md    # Standard ML training patterns
   └── scripts/
         └── model-deploy.py   # Model deployment automation

/Users/{username}/.assistant/skills/
├── personal-workflows/
   ├── SKILL.md                # Workflow overview and best practices
   ├── etl-patterns.md         # Personal ETL best practices
   ├── dashboard-templates.md   # Reusable dashboard patterns
   └── scripts/
         └── pipeline-setup.sh   # Environment setup scripts
```

Genie Code automatically picks up your skills the next time you use it in Agent mode. You can also `@` mention skills to ensure Genie Code uses them.

Best practices

Follow these guidelines to write effective skills:

- Choose the appropriate skill type. Use workspace skills for workflows that benefit many users, such as shared machine learning pipelines or domain-specific processes. Use

user skills for personal workflows that are not relevant to others.

- Keep skills focused. Skills work best when they focus on a single task or workflow. Narrow scope makes it easier for Genie Code to recognize when a skill applies.
- Use clear names and descriptions. A concise, descriptive name and summary help Genie Code match the right skill to the right request.
- Be explicit and example-driven. Describe workflows step by step and include concrete examples or patterns Genie Code can reuse.
- Avoid unnecessary context. Only include information that is required for the task. Extra detail can make skills harder to apply reliably.
- Iterate over time. Treat skills as living workflows. Small updates based on real usage can significantly improve results.
- Separate guidance from automation. Use markdown to explain intent and best practices, and scripts for repeatable actions. Keeping these concerns distinct makes skills easier to maintain and reuse.
- Version control your skills. Back your skills folder with Databricks Git folders to track changes, collaborate with teammates, and roll back when needed.

See also

- Tips to improve Genie Code responses: Learn how to manually reference skills in the chat prompt.
- Agent skills for AI coding assistants: Discover and install agent skills for AI coding assistants like Claude and GitHub Copilot.